

Yapay zeka modelleriyle sohbet ederken bazen her şeyi hatırlıyormuş gibi hissetsek de, aslında her modelin bir "sabır" ve "hafıza" sınırı vardır. Uygulama geliştirme dünyasında biz buna **Bağlam Penceresi (Context Window)** diyoruz.

Eğer token'ları yapay zekanın atomları olarak kabul ettiysek, bağlam penceresini de bu atomların sığabileceği bir **çalışma masası** veya **kısa süreli bellek** olarak düşünebiliriz. Gelin, bu kavramın neden bir uygulamanın kaderini belirlediğini, teknik detaylara boğulmadan ama mantığını kavrayarak inceleyelim.

1. Çift Yönlü Bir Hesaplama: Girdi + Çıktı

Pek çok kullanıcı, sadece kendi yazdığı sorunun (prompt) hafızada yer kapladığını düşünür. Ancak gerçek farklıdır. Bağlam penceresi, etkileşimin her iki tarafını da içine alan bir havuzdur.

- **Prompt (Girdi):** Sizin gönderdiğiniz metin.
- **Completion (Çıktı):** Modelin size verdiği yanıt.

Basit bir matematik yapalım:

Diyelim ki Hippokrates'e ait Latince bir özdeyişi modele gönderdiniz ve çevirisini istediniz.

- Girdi diziniz (Latince metin): **27 Token**
- Modelin yanıtı (Türkçe çeviri): **36 Token**
- **Toplam Tüketim:** $27 + 36 = 63$ Token

Eğer kullandığınız modelin bağlam penceresi çok dar olsaydı (örneğin sadece 100 token), bir sonraki sorunuz için elinizde sadece $100 - 63 = 37$ token'lık bir yer kalacaktı. Bu sınırı aştığınız an, model size "Daha fazlasını kabul edemem" diyecek veya konuşmanın en başını "unutmaya" başlayacaktır.

2. Neden "Sonsuz Hafıza" Yok?

"Neden devasa bir bellek yapmıyoruz?" diye sorabilirsiniz. Cevap tamamen **maliyet ve işlem gücü** ile ilgili. Model, bağlam penceresindeki her bir token ile diğer tüm token'lar arasındaki ilişkiyi (hatırlarsanız "Dikkat Mekanizması" demiştik) sürekli hesaplamak zorunda. Pencere ne kadar büyürse, işlemcinin (GPU) üzerindeki yük de o kadar katlanarak artıyor.

Bu yüzden, uygulama geliştirirken en isabetli modeli seçmek aslında en iyi "bellek yönetimini" seçmekle aynı anlama geliyor.

3. Geliştiriciler İçin Stratejik Önemi

Bağlam penceresini sadece bir sınırlama olarak değil, bir **kaynak yönetimi** olarak görmeliyiz. Bu kavram şu kritik kararları vermemize neden olur:

- **Model Seçimi:** Kısa bir chatbot yapıyorsanız 8k-16k token'lık bir pencere yeterli; koca bir hukuk dosyasını veya kitabı analiz ederseniz 128k, hatta 1M token'lık (Gemini 1.5 Pro gibi) devasa pencerelere ihtiyaç duyarsınız.
- **Dil Tercihi:** Türkçe gibi eklemeli diller daha fazla token tükettiği için, aynı bağlam penceresinde İngilizce bir metnin 10 sayfasını işleyebilirken, Türkçe bir metnin 7 sayfasında limitlere takılabilirsiniz.
- **Hata Yönetimi:** Eğer limitleri takip etmezseniz, uygulamanız en kritik yerde kullanıcıya hata döndürebilir. Bu yüzden iyi bir geliştirici, her zaman arka planda token sayacı çalıştırır.

4. Bağlam Penceresi Dolduğunda Ne Olur?

Hafıza dolduğunda iki ana strateji izlenir:

1. **Sert Durdurma:** Model "Limit doldu, işlem yapamam" der (En istenmeyen senaryo).
2. **Kaydırma (Sliding Window):** Model, yeni gelen token'lara yer açmak için en eski konuşmaları hafızasından siler. Kullanıcının adını en başta söylediği bir sohbette, pencere dolduğunda modelin aniden "Adın neydi?" diye sormasının sebebi tam olarak budur.

Sonuç

Bağlam penceresi, yapay zekanın "dikkatini" koruyabildiği alanın sınırıdır. Uygulamanızın ne kadar "akıllı" veya "unutkan" görüneceği, sizin bu pencereyi ne kadar verimli kullandığınıza bağlıdır.